Proyecto	ControlaTusCuentas
Tipo	Aplicación Web — SPA sin framework
Tecnologías	HTML5 · CSS3 · JavaScript ES2020
API externa	ExchangeRate API (open.exchangerate-api.com)
Persistencia	localStorage (sin backend)
Fecha	Abril - 2026

Índice de contenidos

1. Introducción y objetivos…….. Pág. 3
2. Descripción funcional…….. Pág. 3
3. Arquitectura y diseño…….. Pág. 4
4. Programación Orientada a Objetos…….. Pág. 5
5. Manipulación del DOM…… Pág. 6
6. Consumo de API externa…… Pág. 6
7. Funcionalidades adicionales…….. Pág. 7
8. Interfaz y experiencia de usuario…….. Pág. 8
9. Estructura de archivos…… Pág. 9
10. Conclusiones…… Pág. 9

1. Introducción y objetivos

ControlaTusCuentas es una aplicación web de gestión de finanzas personales desarrollada íntegramente con tecnologías web estándar: HTML5, CSS3 y JavaScript ES2020, sin el uso de frameworks ni librerías externas de terceros. El proyecto demuestra que es posible construir una aplicación completa, funcional y con interfaz cuidada utilizando únicamente las capacidades nativas del navegador.

Los objetivos principales del proyecto son:

- Implementar una arquitectura orientada a objetos clara y extensible mediante herencia ycomposición de clases.
- Manipular el DOM de forma dinámica sin recargar la página, creando una experiencia de usuariofluida.
- Consumir una API REST externa para obtener datos financieros en tiempo real.
- Ofrecer funcionalidades de búsqueda, filtrado y visualización de datos.
- Gestionar la autenticación de usuarios y la persistencia de datos mediante localStorage.
- Proporcionar opciones de exportación de datos en formatos CSV y PDF.
- Diseñar una interfaz completamente responsiva (Mobile First) con soporte de modo oscuro.

2. Descripción funcional

La aplicación se organiza en cuatro páginas principales, cada una gestionada por su propio controlador:

Página	Archivo	Función principal
Inicio	index.html	Conversor de divisas y tipos de cambio en vivo
Login	login.html	Registro e inicio de sesión de usuarios
Registrar	registro.html	Formulario para añadir transacciones
Resumen	resumen.html	Métricas, gráfico, filtros y exportación

2.1 Flujo de usuario

El usuario accede a la página de inicio sin necesidad de autenticarse, donde puede consultar los tipos de cambio en tiempo real y usar el conversor. Un botón "Acceder" en el header despliega un modal de login/registro sin abandonar la página. Una vez autenticado, puede registrar transacciones en cualquier divisa —convertidas automáticamente a EUR— y consultar su resumen financiero con filtros avanzados y opciones de exportación.

2.2 Gestión de divisas

La conversión de monedas utiliza EUR como pivote: cualquier cantidad se convierte primero a euros y desde ahí a la moneda destino. Las transacciones almacenan tanto el importe original como su equivalente en EUR para preservar el valor histórico independientemente de las fluctuaciones del tipo de cambio.

3. Arquitectura y diseño

El proyecto sigue el patrón MVC (Modelo-Vista-Controlador) implementado mediante clases ES6:

Capa	Clases / Módulos	Responsabilidad
Modelo	AuthManager, StorageManager, DivisasService, ApiService	Datos, lógica de negocio, persistencia y comunicación con
Vista	DomManager	Generación de todo el HTML dinámico mediante métodos
Controlador	BaseController y subclases	Coordinación entre modelo y vista, gestión de eventos

3.1 Jerarquía de herencia

API: ApiService → DivisasService

DivisasService extiende ApiService heredando fetchData() y añadiendo la lógica de negocio de divisas: carga de tasas, conversión con pivote EUR y formateo de datos.

Controladores: BaseController → InicioController / RegistroController / ResumenController

BaseController centraliza verificación de sesión, inicialización del tema, inyección del menú de usuario y gestión del modal de login en el header. Cada subclase implementa únicamente la lógica específica de su página.

3.2 Módulos ES6

Todos los archivos JS utilizan ES Modules (import/export), lo que permite separación de responsabilidades clara y carga diferida. No se usa ningún bundler: el navegador resuelve el grafo de dependencias de forma nativa.

4. Programación Orientada a Objetos

El proyecto implementa ocho clases organizadas en dos jerarquías de herencia:

Clase	Hereda de	Propiedades clave	Métodos principales
ApiService	—	baseUrl	fetchData()
DivisasService	ApiService	tasas, base, fechaActualizacion	cargarTasas(), convertir(), obtenerMonedas()
BaseController	—	contenedor, _modoModal	_inicializarBase(), _inicializarModalLogin(), _handleModalLogin()
InicioController	BaseController	servicio (DivisasService)	iniciar(), _cargarDivisas(), _calcularYMostrar()
RegistroController	BaseController	servicio (DivisasService)	iniciar(), _guardar(), _actualizarInfoConversion()
ResumenController	BaseController	_filtros	iniciar(), _aplicarFiltros(), _renderizarGrafico(), _eliminar()
AuthController	—	contenedor, modo	iniciar(), _handleLogin(), _handleRegistro()
DomManager	—	— (métodos estáticos)	htmlMetricas(), htmlGrafico(), crearItemTransaccion()

4.1 Ejemplo de herencia: DivisasService

DivisasService hereda fetchData() de ApiService y lo invoca con la moneda base como parámetro de ruta. La clase hija añade estado propio y dos métodos de negocio clave:

- convertir(cantidad, origen, destino) — conversión con EUR como pivote.
- obtenerTasasDestacadas(monedas) — subconjunto filtrado para el widget de inicio.

5. Manipulación del DOM

Toda la generación de HTML es dinámica. Los archivos HTML contienen únicamente la estructura fija del header, el main vacío (#app) y el footer. DomManager inyecta el contenido mediante innerHTML o appendChild, sin recargar la página.

5.1 Estrategia de renderizado

- **Inyección de estructuras completas:** htmlMetricas(), htmlFormulario(), htmlHero() devuelven cadenas HTML asignadas a contenedor.innerHTML.
- **Creación de elementos individuales:** DomManager.crear(tag, attrs, html) encapsula document.createElement con atributos opcionales.
- **Actualización sin recarga:** _aplicarFiltros() vacía y repobla #lista-transacciones sin tocar el resto de la página.
- **Callbacks en creación:** el botón eliminar de cada transacción recibe su onEliminar al crear el elemento.

5.2 Actualización reactiva

Los eventos de input en los filtros disparan _aplicarFiltros(), que recalcula la lista visible sin tocar las métricas globales. El conversor de divisas actualiza el resultado en tiempo real en cada evento input sobre cantidad y monedas.

6. Consumo de API externa

La aplicación consume la API pública de ExchangeRate para obtener los tipos de cambio:

```
GET https://open.exchangerate-api.com/v6/latest/EUR
```

6.1 Flujo de carga

1. InicioController llama a this.servicio.cargarTasas("EUR").
2. DivisasService.cargarTasas() invoca this.fetchData("EUR") heredado de ApiService.
3. ApiService.fetchData() construye la URL y realiza el fetch con manejo de errores HTTP.
4. La respuesta JSON se almacena en this.tasas, this.base y this.fechaActualizacion.
5. El controlador pobla los selectores, renderiza el grid de tasas y calcula la conversión inicial.

6.2 Manejo de errores

Si la API no responde, ApiService lanza una excepción capturada por el controlador.

DomManager.mostrarError() muestra un mensaje en el área correspondiente y el conversor indica "Servicio no disponible". La aplicación sigue siendo usable en sus demás funcionalidades.

7. Funcionalidades adicionales

7.1 Autenticación simulada

AuthManager implementa un sistema completo sin backend. Las contraseñas se hashean con djb2 antes de almacenarse. Cada usuario tiene sus transacciones aisladas en ctc-transacciones-{id}. La sesión activa se guarda en ctc-sesion sin incluir el hash. BaseController gestiona un modal de login/registro integrado en el header disponible en todas las páginas.

7.2 Sistema de filtrado

filtrarTransacciones() acepta un objeto con cinco dimensiones independientes:

Filtro	Tipo	Comportamiento
texto	string	Búsqueda parcial en categoría, monto y moneda (case-insensitive)
tipo	enum	todos / ingreso / gasto
categoria	enum	todas / Salario / Alimentación / ... / Otros
fechaDesde	date string	Excluye transacciones anteriores a la fecha
fechaHasta	date string	Excluye transacciones posteriores a la fecha

7.3 Visualización de datos

El gráfico de barras agrupadas se genera en HTML/CSS sin canvas ni librerías externas. DomManager.htmlGrafico(datos) calcula la altura de cada barra proporcionalmente al valor máximo y genera un bloque por categoría. Las barras tienen tooltips nativos y respetan el modo oscuro mediante variables CSS.

7.4 Exportación de datos

Formato	Función	Detalle técnico
CSV	exportarCSV()	Blob con BOM UTF-8, descarga directa via URL.createObjectURL()
PDF	exportarPDF()	HTML maquetado en ventana nueva, diálogo de impresión del navegador

Ambas exportaciones respetan los filtros activos. Si no hay transacciones que los cumplan, se muestra un toast informativo en lugar de generar un archivo vacío.

8. Interfaz y experiencia de usuario

8.1 Diseño Mobile First

Los estilos base están escritos para pantallas pequeñas y se amplían con dos breakpoints:

Breakpoint	Ancho	Cambios principales
Móvil (base)	< 480px	Header en dos filas: marca + acciones arriba, navegación abajo
Tablet / Desktop	≥ 480px	Header en una sola fila, tres columnas auto centradas (marca | nav | acciones)
Desktop grande	≥ 768px	Mayor padding, fuentes más grandes, max-width en contenido

8.2 Modo oscuro / claro

ThemeManager gestiona el tema via data-tema en html. Todos los colores son custom properties CSS definidas en :root (modo claro) y sobreescritas en [data-tema="oscuro"]. La preferencia se persiste en localStorage y se inicializa respetando prefers-color-scheme.

8.3 Feedback al usuario

- Toast notifications para confirmación de acciones (guardar, eliminar, exportar).
- Mensajes de error con auto-ocultado a los 4 segundos en formularios de autenticación.
- Indicador de punto parpadeante en el widget de tasas de cambio.
- Botón de eliminar que aparece al hacer hover sobre cada transacción.
- Contador de resultados bajo los filtros cuando la vista está filtrada.
- Estado vacío ilustrado cuando no hay transacciones registradas.

9. Estructura de archivos

Archivo	Tipo	Lín. aprox.	Rol
index.html	HTML	30	Página de inicio
login.html	HTML	25	Autenticación
registro.html	HTML	30	Registro de transacciones
resumen.html	HTML	30	Resumen y exportación
style.css	CSS	1150	Estilos globales + modo oscuro
ApiService.js	JS	20	Cliente HTTP base
DivisasService.js	JS	55	Lógica de divisas
AuthController.js	JS	80	Controlador de login.html
BaseController.js	JS	120	Controlador base compartido
InicioController.js	JS	70	Controlador de index.html
RegistroController.js	JS	75	Controlador de registro.html
ResumenController.js	JS	90	Controlador de resumen.html
DomManager.js	JS	370	Generación de DOM
AuthManager.js	JS	90	Autenticación y sesión
StorageManager.js	JS	110	Persistencia y exportación
ThemeManager.js	JS	70	Modo oscuro/claro

10. Conclusiones

ControlaTusCuentas demuestra que es posible construir una aplicación web completa, mantenible y con una interfaz de calidad utilizando únicamente las APIs nativas del navegador. La arquitectura MVC con ES Modules facilita la separación de responsabilidades y hace que el código sea fácilmente extensible sin la complejidad de un framework.

El uso de herencia entre clases, la gestión reactiva del DOM, el consumo de una API externa real, los filtros multidimensionales, la autenticación con aislamiento de datos por usuario, el modo oscuro basado en custom properties CSS y las opciones de exportación en CSV y PDF conforman un proyecto que cubre de forma completa y cohesionada todos los requisitos establecidos para la entrega.

La decisión de no usar ningún framework supone un reto adicional —gestión manual del estado, renderizado y enrutado— que al mismo tiempo ha permitido una comprensión más profunda del funcionamiento interno de las herramientas que normalmente abstraen estos detalles.